Areli Diaz
05/06/26
Foundations of Python Programming
Assignment05
Mod05-GitHub Link

Assignment 5: Advanced Collections and Error Handling

Introduction

In Module 5, I learned how to work with dictionary data collections, list of dictionaries, JSON files, and structured error handling using try-except constructs. I used these topics in this week's assignment by creating a program that creates and saves a student roster to a JSON file. Additionally, I was introduced to Github and created my first repository where I uploaded my assignment 5 files. I then shared my repository with my classmates for an informal peer review of my work.

Dictionary Data Collections

Dictionaries are data collections that use key:value pairs to organize data. Dictionaries use the {} brackets and have the keys in either single or double quotes. The values can be stored as any data type. Below is an example of a dictionary row that I created for assignment 5:

```
student_data =
      {"FirstName":"Areli","LastName":"Diaz","CourseName":"Python 100"}
```

The keys are labels that give more information about the values which make dictionaries easier to read and understand versus using lists that give no context on what the values mean. Additionally, dictionary keys make it easier to extract values by using the keys instead of remembering the position of the value in a list.

In assignment 5, I created a list of dictionaries similar to the list of lists I created for assignment 4. The table of dictionaries named "students" consists of rows made of dictionaries that store a student's first name, last name, and course name. Every value entered by the user, was assigned to a unique key. For example, my program asked the user to enter the first name of the student and then stored the user's response in the `student_first_name` variable. I then created a dictionary where I assigned the `student_first_name` variable to the "`FirstName`" key. The same was done for the student's last name and course name. Each row in the `students` table is a dictionary organized as shown below:

```
student_data = {"FirstName": student_first_name, "LastName":
       student_last_name, "CourseName": course_name}
```

I also made sure to replace any positional indexing from the starter script with the keys from my dictionaries so that I extract the correct values as shown in Figure 1:

```python
# Present the current data
elif menu_choice == "2":
    print("\nThe current data is:")

    # Convert each row from a list to string data and display comma separated values
    for each_row in students:
        student_csv = f"{each_row[0]},{each_row[1]},{each_row[2]}" # This converts list back into a csv string
        print(student_csv)  # this displays each students name as a comma separated values
    continue
```

```python
# Present the current data
elif menu_choice == "2":

    # Process the data to create and display a custom message
    print("-"*50)
    print('Students currently registered (First Name, Last Name, Course Name): ')
    for student in students:
        print(f"{student["FirstName"]}, {student["LastName"]}, {student["CourseName"]}")
    print("-"*50)
    continue
```

Figure 1 Replacing Positional Indexing with Dictionary Keys

JSON Format

Another useful characteristic of dictionaries is that they are compatible with JavaScript Object Notation (JSON) formats and files. JSONs are like python dictionaries in that they consist of key-value pairs enclosed in {} brackets, can have values of any data type, and have key:value pairs separated by commas. One important difference is that JSON keys need to be enclosed in double quotes. Like dictionaries in Python, JSON is easier to read and understand for humans than the CSV files we have been using because there is more context on what the values represent.

In assignment 5, I made sure that each row in the students table was defined using dictionaries so that they could be written into a JSON file. To do this, I imported the built in json module at the beginning of my script which allows me to decode JSON to Python objects using the `json.load()` function and encode Python objects to JSON files using the `json.dump()` function (json — JSON encoder and decoder — Python 3.14.5 documentation). An example of using the `json.load()` was at the beginning of my script when I wanted to pull the student information that was already in the JSON file and save it to my students table before adding new student information as shown in Figure 2.

```
file = open(FILE_NAME, "r") # Extract the data from the file
students = json.load(file)
file.close()
```

Figure 2 Reading from a JSON file using json.load()

For menu option #3, I used the `json.dump()` function to save the old and new student information to a JSON file. In Figure 3, my script shows how this is easily done by opening the JSON file using the write permission, using the `json.dump()` function to encode my students table to JSON format, write the students data to the JSON file, then lastly, close the file to save the changes. We do not have to go through the process of converting the data into strings to then write into a file like we had to do with CSV files.

```
file = open(FILE_NAME, "w")
json.dump(students, file, indent=2) # saves the new student information into the json file
file.close()
```

Figure 3 Writing to a JSON File Using json.dump()

Structured Error Handling (Try-Except)

A quote that I liked from the Exceptions in Python YouTube Tutorial by Socratica (Exceptions in Python || Python Tutorial || Learn Python Programming), is "If you fail to plan for failure, you are planning to fail as an engineer. This is because if there is a way for code to break then somehow, somewhere, someone will break it." I agree that our code will usually have unexpected bugs that are hidden until we or someone else finds them. However, the bugs that we can predict can be managed using the try-except construct. The try-except blocks are used to customize error messages, instruct Python on how to deal with the error, and prevent the program from crashing when finding an error. The try-except block goes around code that is risky or could have common errors. The try block tests the code, and if there are any errors, the program jumps to the respective except statement. There can be multiple except statements used to cover as many errors as possible. Python also has a list of built in exceptions that give more technical information on the error. You can also have the error display custom messages that make the error information easier to understand and troubleshoot for the user.

For example, at the beginning of my script, I enclosed the code that opens the "Enrollments.json" file with a try-except statement because there are common issues that can happen when opening a file. For example, a user could have the file named incorrectly or not exist at all. If the program runs the code in the try block and finds an error, then it jumps to the except block that addresses that specific error. If the user has the "Enrollments.json" file named incorrectly, the program will not be able to find the file and will raise the `FileNotFoundError` exception. In the `FileNotFoundError` exception block, I included a custom message that tells the user what the error is plus more technical information that might be useful for more

experienced users. I also added the `Exception` error that is nonspecific and can catch any other errors that I didn't account for. I added a finally block that executes regardless of whether there was an error and will make sure that my file closes. An example of a try-except statement in my script is shown in Figure 4.

```
# When the program starts, read the file data into a list of lists of dictionaries
try:
    file = open(FILE_NAME, "r") # Extract the data from the file
    students = json.load(file)
    file.close()
except FileNotFoundError as err: # Gives a file not found error incase the file doesn't exist
    print('File not found, please create file and try again\n')
    print("---Error Details---")
    print(err, err.__doc__, type(err), sep='\n') # Prints more technical detail on the errors
except Exception as err: # This covers any other errors that might happen
    print('Non-specific error found.\n')
    print("---Error Details---")
    print(err, err.__doc__, type(err), sep='\n')
finally:
    if file is not None and not file.closed: # Checks if file has content and is closed, if not then it closes the file
        print('File not closed, closing now')
        file.close()
```

Figure 4 Example of Error Handling Using the try-except Statement

Another use for error handling is to limit the type of response from the user, especially when entering the student's first and last name for menu choice #1. I want to make sure that the user only uses letters, so to add that limitation, I added an if statement that uses the `.isalpha()` function to check the user's input. If there are any numbers or special characters, the if statement raises a `ValueError` message that tells the user to retry entering the information using only letters. Additionally, I enclosed this part of the code in a try block and added an `ValueType` exception that will give the user more information on the error and prevent the program from crashing. The user can then easily retry entering the information instead of restarting the whole program. I decided not to limit the course name because most course names include numbers such as "Python 100" and I didn't want to add that restriction. Figure 5 demonstrates how I used the try-except block to restrict the user's response.

```python
# Input user data
if menu_choice == "1":  # This will not work if it is an integer!
    try:
        student_first_name = input("Enter the student's first name: ")
        if not student_first_name.isalpha(): # Checks to see that the user only used letters
            raise ValueError("Please only use letters when entering student's first name")

        student_last_name = input("Enter the student's last name: ")
        if not student_last_name.isalpha(): # Checks to see that the user only used letters
            raise ValueError("Please only use letters when entering student's last name")

        course_name = input("Please enter the name of the course: ") # no alpha check since course names have #s
        student_data = {"FirstName": student_first_name, "LastName": student_last_name, "CourseName": course_name}
        students.append(student_data)
        print(f"You have registered {student_first_name} {student_last_name} for {course_name}.")

    except ValueError as err: # If the user uses numbers or special characters, an error message will show
        print("---Error Details---")
        print(err, err.__doc__, type(err), sep='\n')
    continue
```

Figure 5 Restricting User Input

The last try-except block that I used in my script was for menu option #3 where I enclosed the `open()`, `json.dump()` and `file.close()` functions and set the possible exceptions as `TypeError` to cover invalid JSON format errors, and the `Exception` error to catch any nonspecific errors.

```python
# Save the data to a file
elif menu_choice == "3":
    try:
        file = open(FILE_NAME, "w")
        json.dump(students, file, indent=2) # saves the new student information into the json file
        file.close()
    except TypeError as err: # Goes through all the possible errors a user might find
        print('Please make sure that data is in valid JSON format')
        print("---Error Details---")
        print(err, err.__doc__, type(err), sep='\n')
    except Exception as err:
        print('Non-specific error found.\n')
        print("---Error Details---")
        print(err, err.__doc__, type(err), sep='\n')
    finally: # Makes sure the file closes even if there was no error
        if file is not None and not file.closed:
            print('File not closed, closing now')
            file.close()

    print("The following data was saved to file!")
    for student in students:
        print(f"{student["FirstName"]}, {student["LastName"]}, {student["CourseName"]}")
    continue
```

Figure 6 Menu Choice #3 Error Handling Exceptions

Learning to use exceptions has helped me think through different situations where my code could fail by looking at my program from the perspective of different users with varying experience in programming. This helps manage the errors that could arise and make the program more user friendly so that users can understand the errors and easily correct them.

Creating a GitHub Account

GitHub is a cloud-based platform that allows developers to share, store, and manage their scripts and files. It is a collaborative space that helps developers find resources, share their work, and store their files. I have heard about Github many times before but never understood what it was, so I was excited to create my account and start exploring. To create and account, I went to https://github.com/, clicked on create an account, realized that I had already made an account years ago but didn't remember, then proceeded to login.

I then created my Assignment 5 repository by clicking on the New Repository button and filled out the information as shown below:

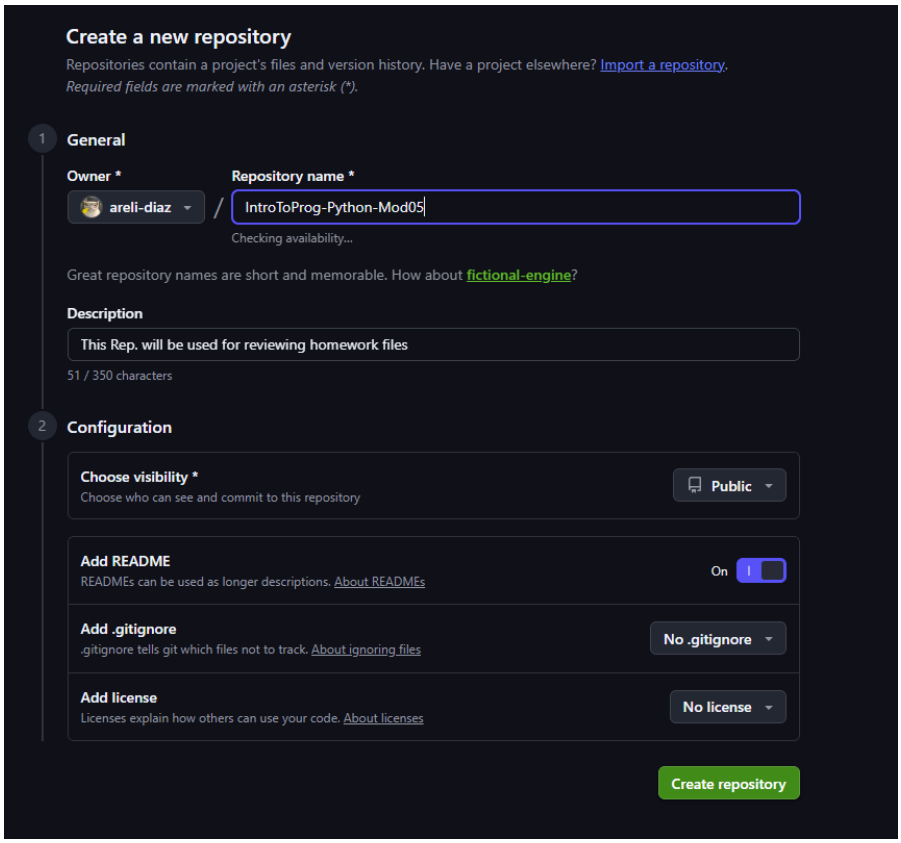

Figure 7 Creating Assignment 5 Repository on GitHub

After creating my first repository, I then uploaded my Assignment 5 script and my knowledge document to finish setting up my first repository.

After logging in and creating my repository, I began to explore my homepage and lookup tutorials on how to use the platform. What I began to realize is that GitHub can be used like a LinkedIn profile in that it is used by recruiters and hiring managers to gauge your skills as a developer. Building your GitHub profile, consistently contributing, and earning rewards are tools that can help you stand out to recruiters. I'm excited to continue using GitHub and start building my profile.

Summary

Module 5 introduced me to topics covering the Python dictionary data collection, working with JSON files, and learning to manage errors using the try-except statements. These topics helped me successfully create and run my assignment 5 program that reads and saves information from a JSON file to a list of dictionaries, presents the user with menu choices, allows the user to add student information to a roster, presents the current roster list, saves the roster to a JSON file, and easily exits the program. This program also has structured error handling that catches and presents errors instead of crashing the program. I also created my first GitHub repository, uploaded my assignment 5 documents, and shared my work with my classmates.

References

json — JSON encoder and decoder. (n.d.). Retrieved from docs.python.org: https://docs.python.org/3/library/json.html

Root, R. (2026). Introduction to Programming with Python: Module 04 – Collections of data. Seattle, Washington, USA.

Root, R. (n.d.). Introduction to Programing with Python, Module 03 - Programming Tools and Techniques. Seattle, Washington, USA.

Root, R. (n.d.). Introduction to Programing with Python: Module 05 - Advanced Collections and Error Handling. Seattle, Washington, USA.

Root, R. (n.d.). Introduction to Programming with Python, Module 01-Python Basics. Seattle, Washington, USA.

Root, R. (n.d.). Introduction to Programming with Python, Module 02 - Programming Basics. Seattle, Washington, USA.

Socratica. (2018, June 2). Exceptions in Python || Python Tutorial || Learn Python Programming. Retrieved from https://youtu.be/nlCKrKGHSSk?si=cNV9wilT3dH1ANy4